PNI マイグレーション実行計画レポート

本レポートは、レガシー請求処理 Spring Boot API サービス `legacy-billing` (`0.0.1-SNAPSHOT` / **ベース監査** `wf-da941741`: **23/100 Grade D**) に対する PNI 移行設計ワークフロー `java_api_spring_messy_migration`（トポロジカル依存 Wave 抽出・暗黙 API 契約復元・状態/DB/HTTP 境界タッチポイント解析・OpenRewrite レシピ照合・仕様/バグ境界分離・ライブ DAST 検証ゲート設計）に基づき策定された、**Java 21 LTS + Spring Boot 3.4 + PostgreSQL 16 + Redis 7 + Clean Architecture** への段階的移行ブループリントである。



legacy-billing マルチ軸マイグレーション実行計画サマリカード

確定済み前提条件（Confirmed Assumptions）と非目標（Non-Goals）

- **移行モード / ロールアウト戦略**: `architectural_restructure` / `phased_pr`（**単一リポジトリ段階的 PR 方式**）・外部 API 互換ポリシー `strict`・仕様曖昧性ポリシー `ask_human`
- **確定済み技術スタック**: **Java 21 LTS** + **Spring Boot 3.4.x BOM**（Jakarta EE 10 / JUnit 5 Jupiter / JaCoCo）+ **PostgreSQL 16**（Spring Data JPA + Flyway）+ **Redis 7**（TTL 15分キャッシュ外部化）+ **Clean Architecture**（`domain` / `port` / `usecase` / `adapter`）
- **確定済み外部契約 & DAST ガードレール**: `POST /charge` の既存 JSON キー（`customerId`, `amountCents`, `billedCents`）を厳格維持し、`String ==` 税率判定・

`100L` 固定リトライ・`controllerPing` 診断キー等の Hyrum の法則リスクは Phase 3 前にステークホルダー合意を経る。また `.pni/pentests/examples/java/api_spring_messy/` の 4 プローブスイート（`CWE-306 / CWE-650 / CWE-20 / CWE-693`）全件 PASS を Phase 3/4 の必須セキュリティゲートとする。

- **非目標（Non-Goals）**: マイクロサービス分割、Kafka / RabbitMQ 等の非同期メッセージング基盤導入、および外部決済プロトコル自体の刷新は本移行スコープ外とする（単一デプロイ可能な Maven アーティファクトを維持）。

Table of Contents

1. エグゼクティブサマリとマルチ軸移行戦略

2. As-Is 依存グラフ解析・移行 Wave・暗黙の外部契約

3. To-Be ターゲットアーキテクチャ設計（Clean Architecture & Port/Adapter）

4. 機械的変換（OpenRewrite Codemod）と手動ブロッカー分離計画

5. 「仕様かバグか」ステークホルダー意思決定ログ（Decision Log）

6. PR 単位の実行 WBS と検証・受け入れゲート

1. エグゼクティブサマリとマルチ軸移行戦略

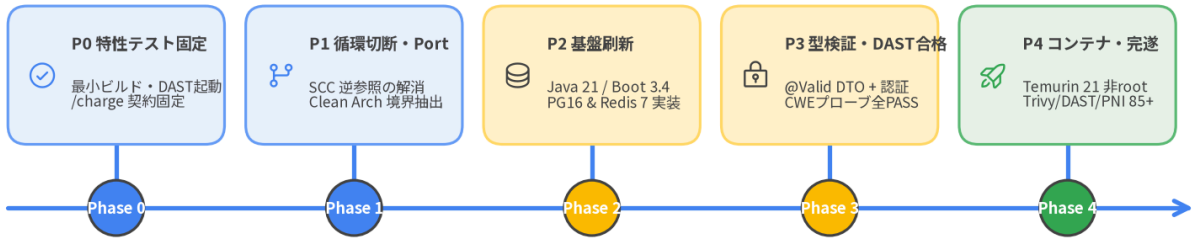
`legacy-billing` は現状 PNI 健全性診断（`wf-da941741`）で **23 / 100（Grade D・指摘24件）** と評価されており、`pom.xml` の Java 1.8 指定と `spring-boot-starter-web:LATEST`（Spring 6 / class file v61）および Java 11 `HttpClient` の衝突によるビルド崩壊（これに起因して `SEC-06 web_pentest` 隔離ループバック DAST も `status=error` で起動停止）、`BillingManager <-> BillingController` の双方向循環依存、プロセス内 `static Map INVOICE_CACHE`、および秘密鍵ハードコード・無認証 `/charge` 境界を抱えている。本計画では「一括ビッグバン書き換え」を避け、**7つの移行軸**を **Phase 0〜Phase 4 の5段階ゲート**へ分解して安全に移行する。

変換軸	As-Is (現状の課題)	To-Be (目標仕様)	移行アプローチ / リスク
言語・ランタイム	Java 1.8 指定 + Java 11 API 混在 Date / SimpleDateFormat / Vector	Java 21 LTS 統一 不変 record / java.time (UTC)	Phase 0 整合復旧 → Phase 2 OpenRewrite 自動変換 [HIGH]
フレームワーク	Spring Boot LATEST (javax.*) フィールド DI / JUnit 4 無検証	Spring Boot 3.4 BOM (jakarta.*) コンストラクタ DI / JUnit 5	OpenRewrite 専用 PR 分離 MockMvc + JaCoCo 検証 [HIGH]
設計思想・構造	Controller <-> Manager 循環依存 計算・I/O・状態が単一クラス密結合	Clean Architecture 4層分離 domain / port / usecase / adapter	Phase 1 逆参照切断 → Wave 順 ArchUnit で依存方向強制 [CRIT]
DB・永続化	spring-tx 欠落・無効 @Transactional 監査ログが標準出力のみで揮発	PostgreSQL 16 + Spring Data JPA Flyway V1 バージョン管理 DDL	BillingAuditRepositoryPort 抽出 Testcontainers PG16 検証 [HIGH]
キャッシュ・状態	static raw Map INVOICE_CACHE Controller 可変フィールド channel	Redis 7 (TTL 15分・JSON 形式) 完全ステートレス Controller	InvoiceCachePort アダプタ化 リクエスト DTO スコープ化 [HIGH]
認証・通信保安	GATEWAY_SECRET 直書き・Timeout 無 /charge CWE-306/650/20・DAST停止	@ConfigurationProperties 外部注入 POST + Security + DAST全件PASS	Phase 2 Outbound 保安化 → Phase 3 DAST プローブ検証 [CRIT]
コンテナ・CI	openjdk:8 root 実行・Healthcheck 無 pull_request_target 特権 CI	Temurin 21 マルチステージ非 root 最小権限 pull_request + SHA 固定	Phase 4 インフラ刷新 + Trivy / web_pentest ゼロ違反証明 [MED]

As-Is -> To-Be マルチ軸ギャップ分析マトリクス (全7変換軸)

5フェーズ段階的実行ゲート（Phase 0 → Phase 4）

構造リファクタリング・機械的アップグレード・仕様バグ修正を同一 PR に混在させると、回帰発生時の原因切り分けが不可能になる。そのため以下の **5段階ゲート方式** を厳守する。



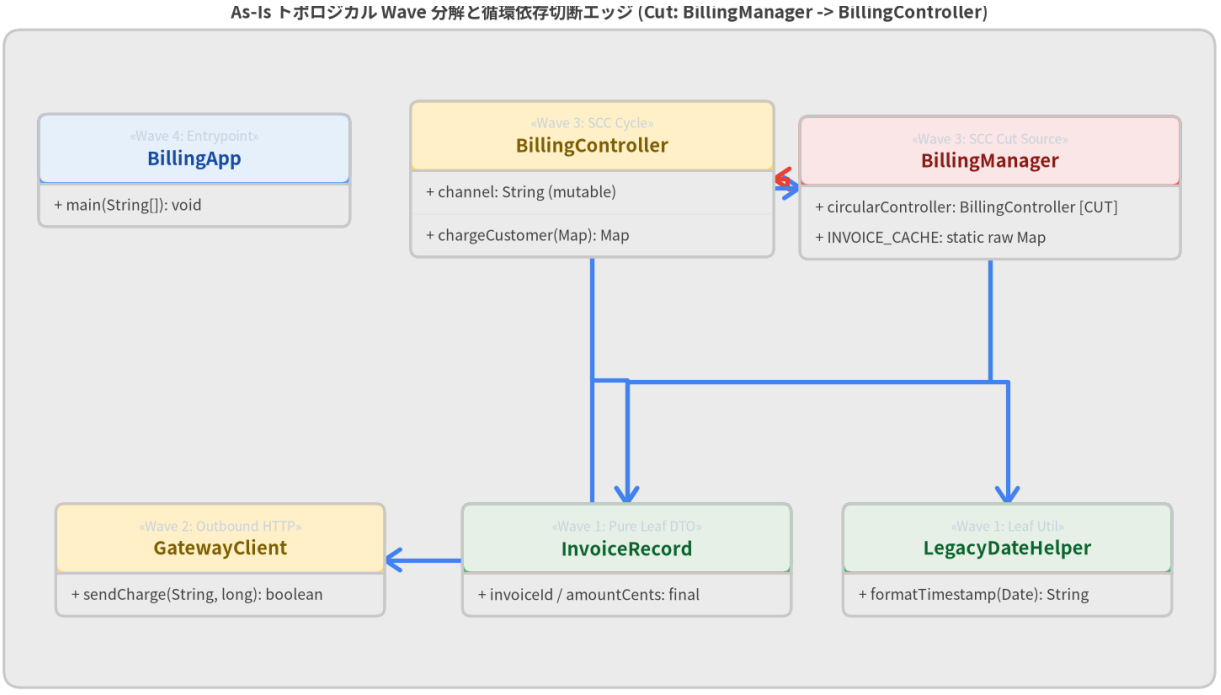
5フェーズ段階的マイグレーション実行ゲート (Phase 0 -> Phase 4)

- **Phase 0（最小グリーンビルド復旧 & Golden Master 固定）**: ロジックには触れず `pom.xml` のバージョン衝突を最小解消して隔離ループバック起動（`web_pentest` 実行基盤）をアンブロックし、`POST /charge` の現行 JSON 挙動をスナップショットテストで固定する。
- **Phase 1（循環依存切断 & Domain / Port 抽出）**: 挙動 100% 保全のまま `BillingManager -> BillingController` 逆参照を切断し、フレームワーク非依存の `domain` / `port` / `usecase` を確立する。
- **Phase 2（OpenRewrite 機械変換 & PostgreSQL / Redis アダプタ実装）**: Codemod 専用 PR（Java 21 / Spring Boot 3.4 / JUnit 5）を先行マージ後、JPA + Flyway・Redis TTL・Timeout 付き Gateway アダプタを実装する。
- **Phase 3（型安全 DTO・認証保護・DAST 合格 & PO 合意仕様反映）**: ステークホルダー承認済みの Decision Log（税率 `.equals()` 比較・リトライ規約等）と `@PostMapping("/charge")` + `@Valid` DTO + Spring Security を導入し、`.pni/pentests/examples/java/api_spring_messy/` の `CWE-306 / CWE-650 / CWE-20 / CWE-693` プローブを全件 PASS させる。

- **Phase 4（コンテナ・CI ハードニング & 最終監査）**：`eclipse-temurin:21` 非 root マルチステージ化と GitHub Actions 最小権限化を完了し、Trivy・ライブ DAST（`web_pentest`）・PNI 最終ゲート（Grade A `85+`）を通過させる。

2. As-Is 依存グラフ解析・移行 Wave・暗黙の外部契約

`migration_analyze` による AST 内部 import グラフおよび強連結成分（Tarjan SCC）分解の結果、全6本番モジュールは **Wave 1（純粋 Leaf）→ Wave 2（外部 HTTP クライアント）→ Wave 3（SCC 循環依存ペア）→ Wave 4（起動エントリポイント）** の4段階に分類された。



トポロジカル移行 Wave (Wave 1-4) と SCC 循環依存の推奨切断ポイント

- **循環依存の推奨切断ポイント（Cycle Cut）**：下位レイヤーである `BillingManager` が上位 Web レイヤーの `BillingController.lastChannel()` を逆参照するバックエッジを切断対象とする。`channel` スカラー値をリクエストスコープの `ChargeCommand` 引数として上位から順方向に渡すことで、挙動を変えずに SCC サイクルを解消できる。

暗黙の API 入出力契約・環境変数インベントリ（後方互換ガードレール）

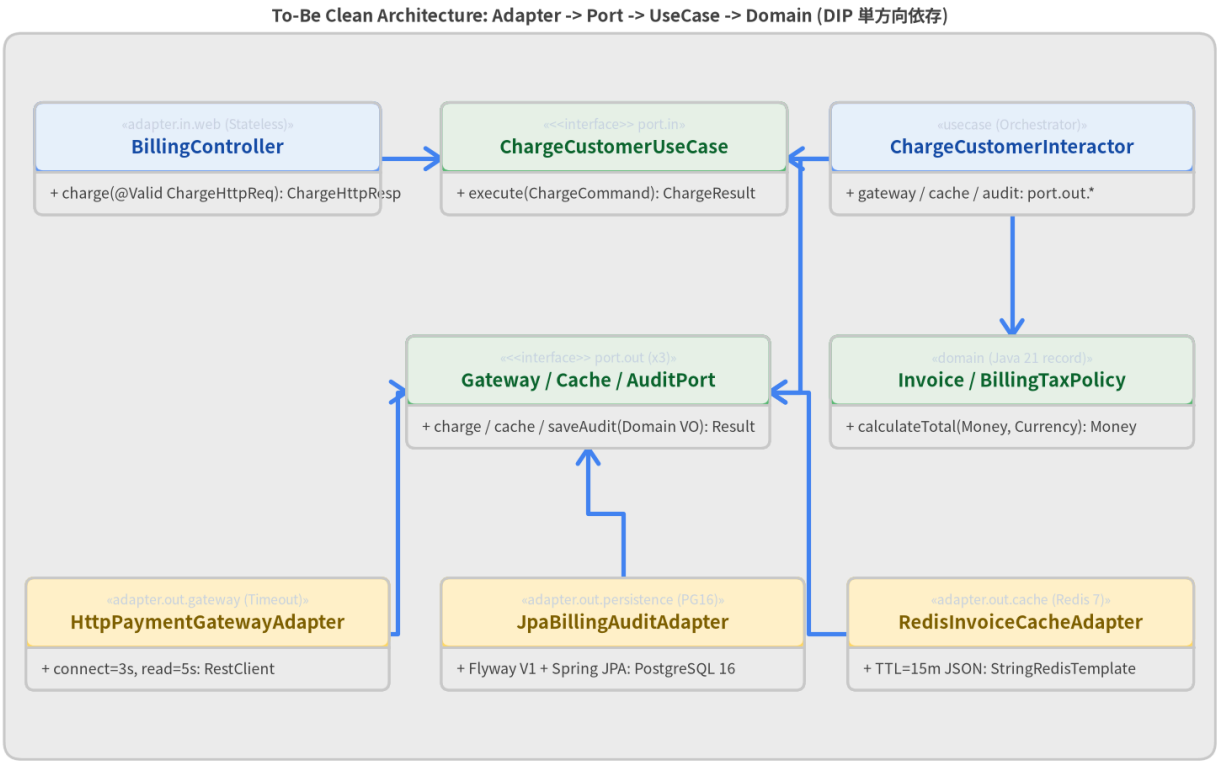
`BillingController.chargeCustomer` は型付き DTO ではなく `Map<String, Object>` で入出力を扱っているため、AST 解析により `getOrDefault` / `put` 呼び出しから実際の外部 JSON 契約を復元した。`api_compatibility: strict` に従い、Inbound Web Adapter の Record DTO で下表のキー互換を保証する。

区分 / 方向	キー・変数名	推定型 / 既定値	抽出箇所	To-Be DTO・設定マッピング方針
POST /charge Input JSON	customerId	String (既定: "anon")	BillingController :L26	ChargeHttpRequest.customerId (Phase 0-2 互換維持 → Phase 3 @NotBlank)
POST /charge Input JSON	amountCents	Integer / Long (既定: "500")	BillingController :L27	ChargeHttpRequest.amountCents (Phase 3 で @Positive 検証を PO 合意適用)
POST /charge Output JSON	customerId	String	BillingManager :L86	ChargeHttpResponse.customerId (厳格互換維持: 必須出力フィールド)
POST /charge Output JSON	billedCents	Long (integer)	BillingManager :L87	ChargeHttpResponse.billedCents (厳格互換維持: 税計算反映後の請求額)
POST /charge Output JSON	controllerPing	String (診断キー)	BillingManager :L81	Decision Log DEC-03 対象: リクエスト X-Channel 由来で互換出力し v2 で廃止予告
Environment Variable	LEGACY_PAYMENT_URL	URI String (fallback URL あり)	GatewayClient :L16	PaymentGatewayProperties (billing.gateway.url) 起動時 @NotNull @URL バリデーション強制

AST 抽出された暗黙の外部 REST 契約 (/charge) と環境変数契約

3. To-Be ターゲットアーキテクチャ設計（Clean Architecture & Port/Adapter）

確定した設計方針に基づき、すべての技術的詳細（Spring Web、PostgreSQL JPA、Redis 7、決済 HTTP 通信）を外側の `adapter` 層へ押し出し、ビジネスルールを純粋 Java 21 `record` で構成された `domain` 層と `usecase` 層へ集約する **Clean Architecture（4層構造）** を導入する。すべての依存矢印は内側（`adapter -> usecase -> port -> domain`）を向き、ArchUnit テストで逆方向 import を CI ブロックする。



To-Be Clean Architecture クラス・Port/Adapter 境界設計図

境界タッチポイント（状態・DB・外部通信）から To-Be コンポーネントへのマッピング

`migration_analyze` が特定した4箇所の境界タッチポイント（`BillingController.channel`、`BillingManager.INVOICE_CACHE`、`@Transactional`

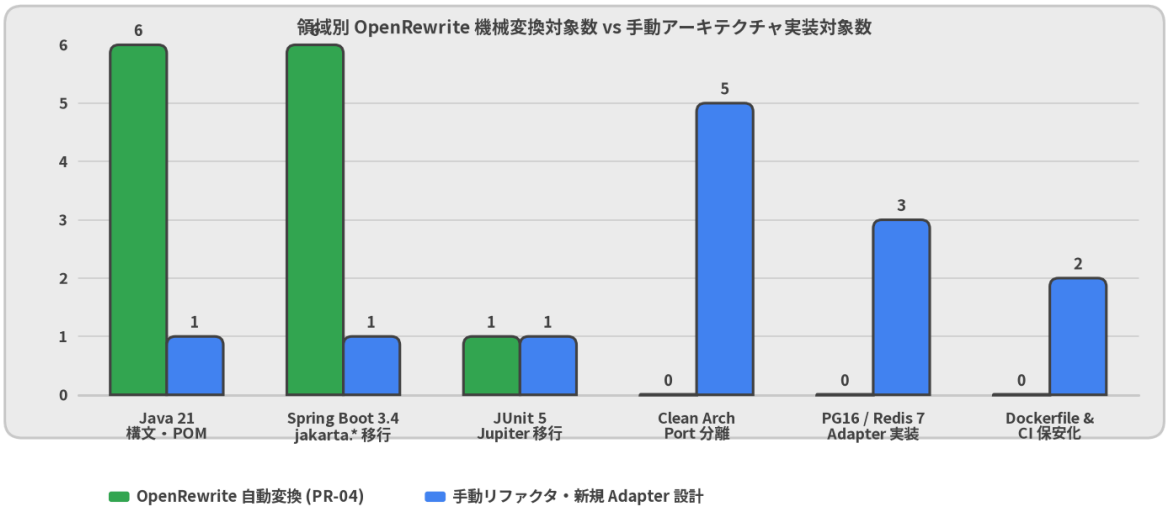
`saveAuditTrail`、`GatewayClient` の `HttpClient`）を含む全既存シンボルの移行先責務を下表に定義する。

As-Is ソース / シンボル	To-Be レイヤー	To-Be コンポーネント / Port 役割	境界分離・モダナイゼーション詳細
InvoiceRecord.java (InvoiceRecord)	domain	domain.model.Invoice [Java 21 record VO]	不変 record 化 + CurrencyCode / InvoiceStatus Enum 導入。フレームワーク依存ゼロ。
LegacyDateHelper.java (formatTimestamp)	domain	domain.support.AuditClock [Pure Time Policy]	非スレッドセーフな SimpleDateFormat と Raw Vector を廃止し java.time.Instant へ統一。
BillingManager.java (processBatchBilling)	usecase / port.in	ChargeCustomerInteractor -> ChargeCustomerUseCase	Controller 逆参照を削除。3つの Outbound Port をコンストラクタ注入する純粋調停層へ再編。
BillingController.java (channel / chargeCustomer)	adapter.in.web	BillingController [Stateless Web Adapter]	可変フィールド channel を完全撤去。@PostMapping + @Valid Record DTO で外部 JSON 契約を維持。
BillingManager.java (INVOICE_CACHE: static Map)	adapter.out.cache -> port.out	RedisInvoiceCacheAdapter -> InvoiceCachePort	プロセス内 static HashMap を撤去し Redis 7 (TTL 15分・JSON 直列化) へマルチノード共有化。
BillingManager.java (@Transactional saveAuditTrail)	adapter.out.persistence -> port.out	JpaBillingAuditAdapter -> BillingAuditRepositoryPort	PostgreSQL 16 + Spring Data JPA + Flyway V1 マイグレーションで請求・監査証跡を永続化。
GatewayClient.java (HttpClient / SECRET)	adapter.out.gateway -> port.out	HttpPaymentGatewayAdapter -> PaymentGatewayPort	秘密鍵を @ConfigurationProperties 注入へ外部化し connect 3s / read 5s タイムアウトを強制。

As-Is クラス/タッチポイントから To-Be Clean Architecture への完全マッピング表

4. 機械的変換（OpenRewrite Codemod）と手動ブロッカー分離計画

マイグレーション失敗の最大要因は、「AST ツールで 100% 機械的に置換できる変更（`javax.* -> jakarta.*`、JUnit 4 → 5 等）」と「人間の設計判断を伴う変更（循環依存の切断、Redis/PostgreSQL アダプタ設計）」が同一 PR の巨大な Diff に混在することである。本計画では、**OpenRewrite による機械的変換を `PR-04` に完全隔離**し、その前段で手動ビルドブロッカー（`BLK-01〜BLK-03`）を解消する。



変換領域別 自動 Codemod 適用ファイル数 vs 手動設計変更ファイル数

OpenRewrite 適用レシピと事前解消必須ブロッカー（BLK-01〜03）

OpenRewrite は「対象プロジェクトがコンパイル可能であること」を前提に型解決 AST 変換を実行し、さらに動的ペネトレーション検証（`web_pentest`：`.pni/pentests/examples/java/api_spring_messy/`）もコンテナ起動成功を前提とするため、**Phase 0（`PR-01`）での最小グリーンビルド復旧**および **Phase 1（`PR-03`）での循環依存切断**を完了してから Phase 2 冒頭（`PR-04`）で下記3レシピを一括実行する。

種別 / ID	レシピまたはブロッカー項目	影響範囲	実行タイミングと解消手順
Blocker BLK-01/04 [CRIT]	コンテナ・Maven ビルド崩壊 & DAST 停止 (Java 1.8 vs Spring v61 / HttpClient / tx)	pom.xml / GatewayClient	Phase 0 (PR-01): target と BOM を整合させ mvn test と web_pentest 隔離ループバック起動を先に復旧する。
Blocker BLK-02/03 [HIGH]	BillingController <-> BillingManager 双方向 @Autowired 循環依存	Controller & BillingManager	Phase 1 (PR-03): Spring Boot 3.x では循環参照が既定禁止のため、Codemod 実行前に逆参照を切断する。
OpenRewrite Recipe 1	org.openrewrite.java.migrate. UpgradeToJava21	本番 Java 6 ファイル + pom.xml	Phase 2 (PR-04): Java 21 LTS コンパイラ設定、非推奨 JDK API 置換、型推論モダナイズを機械適用する。
OpenRewrite Recipe 2	org.openrewrite.java.spring.boot3. UpgradeSpringBoot_3_4	本番 Java 6 ファイル + pom.xml	Phase 2 (PR-04): javax.servlet.* -> jakarta.servlet.* 置換、Spring Boot 3.4 BOM 更新、コンストラクタ DI 化。
OpenRewrite Recipe 3	org.openrewrite.java.testing.junit5. JUnit4to5Migration	BillingManagerTest (テスト 1 ファイル)	Phase 2 (PR-04): org.junit.Test を JUnit 5 Jupiter API へ機械変換（アサーション拡充は手動 PR で実施）。

OpenRewrite 自動変換レシピ一覧と事前解消必須ブロッカー (BLK-01 - BLK-04)

5. 「仕様かバグか」ステークホルダー意思決定ログ（Decision Log）

レガシー移行において最も危険なのは、開発者や AI が「明らかなバグに見えるコード」をインフラ刷新と同時に黙って修正し、その挙動に依存していた外部クライアントを破壊することである（**Hyrum の法則**）。`bug_handling_policy: ask_human` に従い、AST 解析で抽出された下記4件の境界挙動は **Phase 0〜2 では現行挙動を Golden Master テストで厳格維持**し、プロダクトオーナー承認後に **Phase 3（`PR-08` / `PR-09`）で意図的に切り替える**。



Hyrum の法則を防ぐ「仕様 vs バグ」分離・承認ゲートフロー

ステークホルダー承認対象リスト（DEC-01 〜 DEC-04）

ID / 箇所	現行コードの観測挙動 (As-Is)	無断変更時の影響リスク	推奨方針 (Recommended) と選択肢
DEC-01 BillingManager :L52-L57	status == "PAID" / currency == "USD","JPY" を参照等価 (==) 比較しており税加算が実質不発。	.equals() 化すると USD 8% / JPY 10% 課税が急に有効化され、顧客請求額 (billedCents) が変動する。	[推奨 Option A] Phase 0-2 は維持し、PO 承認・事前告知のうえ Phase 3 (PR-09) で Enum 税計算化 [Option B] フラグ配下で段階有効化
DEC-02 BillingManager :L67	amountCents <= 0 のリトライ経路で固定リテラル 100L ($1 相当) を GatewayClient へ課金送信。	即時 400 エラー化すると、$1 与信プローブとして利用中の下流決済連携が失敗する恐れがある。	[推奨 Option A] アクセスログ監査後、不要なら @Positive 検証で HTTP 400 拒否へ統一 [Option B] /charge/verify-card へ分離
DEC-03 BillingManager :L81-L83	レスポンスに controllerPing 診断キーを含め、例外時は空 catch で握り潰して正常応答を返す。	循環依存切断時にキーを削除すると厳格な JSON スキーマ検証を行うクライアントがパース失敗する。	[推奨 Option A] strict 互換に従いリクエスト X-Channel 由来で controllerPing を互換出力維持 [Option B] 利用ゼロ確認後に即時削除
DEC-04 BillingController :L24-L27	GET 含む全 HTTP メソッドを受け付け customerId="anon", amount=500 へ暗黙フォールバックする。	POST 限定化・必須入力化により、GET 死活監視や省略呼び出しを行う旧スクリプトが 405/400 となる。	[推奨 Option A] POST /charge 限定化 (死活監視は /actuator/health) + 必須入力 ProblemDetail 400 [Option B] DTO 既定値で一時フォールバック維持

Phase 3 着手前に合意必須のステークホルダー意思決定ログ (DEC-01 - DEC-04)

6. PR 単位の実行 WBS と検証・受け入れゲート

5フェーズ・7変換軸の全作業を、1 PR あたりレビュー容易な粒度（`S`〜`M` サイズ）かつトポロジカル依存順の **計10 PR（`PR-01`〜`PR-10`）** に分割する。`PR-05`（Gateway 保安化）、`PR-06`（PostgreSQL + Flyway）、`PR-07`（Redis キャッシュ）はすべて `PR-04`（OpenRewrite 完了）のみに依存するため、**3並列で開発・レビューが可能**である。

PR / Phase	規模 / 種別	PR タイトルと主要変更内容	依存 PR	必須検証コマンド (CI Gate)
PR-01 Phase 0	S / Manual	最小グリーンビルド・DAST起動復旧と /charge Golden Master 特性テスト追加	なし	mvn clean test (server_booted=true)
PR-02 Phase 1 (W1)	S / Manual	Wave 1 ドメインモデル抽出 (Invoice record / Money / java.time)	PR-01	mvn test -Dtest=*Domain*,*Invoice*
PR-03 Phase 1 (W3)	M / Manual	Controller<->Manager 循環依存切断と Clean Arch Port / UseCase / ArchUnit 導入	PR-02	mvn test (SCC cycle = 0 & ArchUnit)
PR-04 Phase 2 (W1)	M / Codemod	OpenRewrite 専用 PR: Java 21 LTS + Spring Boot 3.4 (jakarta.*) + JUnit 5	PR-03	mvn rewrite:run && mvn clean test
PR-05 Phase 2 (W2)	S / Manual	HttpPaymentGatewayAdapter 実装 (@ConfigurationProperties + Timeout 3s/5s)	PR-04	mvn test -Dtest=*Gateway* (WireMock)
PR-06 Phase 2 (W2)	M / Manual	PostgreSQL 16 + Spring Data JPA + Flyway V1 永続化アダプタ実装	PR-04	mvn verify -Dtest=*JpaBillingAudit*
PR-07 Phase 2 (W2)	S / Manual	Redis 7 キャッシュアダプタ (TTL 15m) と static INVOICE_CACHE の完全撤去	PR-04	mvn verify -Dtest=*RedisInvoiceCache*
PR-08 Phase 3 (W4)	M / Manual	BillingController ステートレス化・@Valid DTO・Spring Security & DAST CWE 防御	PR-05,06,07	mvn test && pni ... web_pentest (PASS)
PR-09 Phase 3 (W3)	S / Manual	PO 合意済み Decision Log (DEC-01〜04) 税率 Enum 判定・リトライ規約反映	PR-08	mvn clean verify jacoco:check (>=85%)
PR-10 Phase 4 (W4)	S / Hybrid	Temurin 21 非 root マルチステージ Docker ・CI 最小権限化・OpenAPI/README 整備	PR-09	docker build && trivy && web_pentest

トポロジカル順序付き PR 実行 WBS (PR-01 - PR-10) と自動検証コマンド

フェーズ完了受け入れ基準（Exit Criteria）とロールバック戦略

- **Phase 0 完了条件**: `mvn clean test` が Exit Code 0 で完走し、隔離ループバック DAST（`web_pentest`：`server_booted=true`）が起動可能になり、`/charge` の現行 JSON 入出力スナップショットテストが 100% PASS すること。
- **Phase 1 完了条件**: AST 内部 import グラフの循環依存が `0` 件となり、ArchUnit テストにより `domain` / `usecase` 層のフレームワーク非依存が CI 保証されること。
- **Phase 2 完了条件**: Testcontainers 上の PostgreSQL 16（Flyway V1 適用）および Redis 7（TTL 15分失効確認）統合テストが全件 PASS し、ソース内ハードコード秘密鍵および `static Map` が `0` 件であること。
- **Phase 3 & Phase 4 最終受け入れゲート**: JaCoCo カバレッジ **85% 以上**、`.pni/pentests/examples/java/api_spring_messy/` のライブ DAST ペネトレーション検証が **全件 PASS（`failed_probes=0`：無認証 `401/403` `CWE-306`・`PUT /charge -> 405` `CWE-650`・不正数値 `-> 400` `CWE-20`・`nosniff` ヘッダ `CWE-693`）**、Trivy スキャンで `CRITICAL` / `HIGH` **指摘 0 件**、および PNI 健全性再診断（`java_api_spring_messy`）で **Grade A（85点以上）** を達成すること。万一デプロイ後に異常が検知された場合は、イミュータブルなコンテナタグ切り戻しと Flyway 後方互換スキーマ維持により無停止ロールバックを実行する。